

Lab_Assignment_11.2

AI Assisted Coding B37 2303A52077

Task-1 – (Stack Using AI Guidance)

- **Task:** With the help of AI, design and implement a Stack data structure supporting basic stack operations.

Expected Output:

- A Python Stack class supporting push, pop, peek, and empty-check operations with proper documentation.

Prompt:

#write a python class called stack that implements a stack data structure using a list. The class should have the following methods:

#- push(item): adds an item to the top of the stack

#- pop(): removes and returns the item at the top of the stack. If the stack is empty, it should raise an exception.

#- peek(): returns the item at the top of the stack without removing it. If the stack

#- display(): prints the contents of the stack from top to bottom.

#- overflow(): checks if the stack is full. For this implementation, we will assume the stack has a maximum size of 10.

#- underflow(): checks if the stack is empty.

Code:

```
class Stack:
```

```
    def __init__(self):
```

```
        self.stack = []
```

```
        self.max_size = 10
```

```
    def push(self, item):
```

```
        if self.overflow():
```

```
            raise Exception("Stack overflow: cannot add item,  
stack is full.")
```

```
        self.stack.append(item)
```

```
    def pop(self):
```

```
        if self.underflow():
```

```
            raise Exception("Stack underflow: cannot remove  
item, stack is empty.")
```

```
        return self.stack.pop()
```

```
    def peek(self):
```

```
        if self.underflow():
```

```
            raise Exception("Stack underflow: cannot peek item,  
stack is empty.")
```

```
    return self.stack[-1]
```

```
def display(self):
```

```
    print("Stack contents (top to bottom):")
```

```
    for item in reversed(self.stack):
```

```
        print(item, end=' ')
```

```
    print()
```

```
def overflow(self):
```

```
    return len(self.stack) >= self.max_size
```

```
def underflow(self):
```

```
    return len(self.stack) == 0
```

```
# Example usage:
```

```
if __name__ == "__main__":
```

```
    stack = Stack()
```

```
    stack.push(1)
```

```
    stack.push(2)
```

```
    stack.push(3)
```

```
    stack.display()
```

```
    print("Top item:", stack.peak())
```

```
    print("Popped item:", stack.pop())
```

stack.display()

print("Is stack empty?", stack.underflow())

print("Is stack full?", stack.overflow())

```
9  class Stack:
10  def __init__(self):
11      self.stack = []
12      self.max_size = 10
13
14  def push(self, item):
15      if self.overflow():
16          raise Exception("Stack overflow: cannot add item, stack is full.")
17      self.stack.append(item)
18
19  def pop(self):
20      if self.underflow():
21          raise Exception("Stack underflow: cannot remove item, stack is empty.")
22      return self.stack.pop()
23
24  def peek(self):
25      if self.underflow():
26          raise Exception("Stack underflow: cannot peek item, stack is empty.")
27      return self.stack[-1]
28
29  def display(self):
30      print("Stack contents (top to bottom):")
31      for item in reversed(self.stack):
32          print(item, end=' ')
33      print()
34
35  def overflow(self):
36      return len(self.stack) >= self.max_size
37
38  def underflow(self):
39      return len(self.stack) == 0
40
```

Output:

```
11.2.1.py > ...
41  # Example usage:
42  if __name__ == "__main__":
43      stack = Stack()
44      stack.push(1)
45      stack.push(2)
46      stack.push(3)
47      stack.display()
48      print("Top item:", stack.peek())
49      print("Popped item:", stack.pop())
50      stack.display()
51      print("Is stack empty?", stack.underflow())
52      print("Is stack full?", stack.overflow())
53

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS D:\Subjects\ai assisted coding\ai_assi_codes\11.2> & C:/Users/A
assisted coding/ai_assi_codes/11.2/11.2.1.py"
Stack contents (top to bottom):
3 2 1
Top item: 3
Popped item: 3
Stack contents (top to bottom):
2 1
Is stack empty? False
Is stack full? False
PS D:\Subjects\ai assisted coding\ai_assi_codes\11.2>
```

Explanation:

The above code defines a Stack class that implements a stack data structure using a list. It includes methods for pushing, popping, peeking, displaying the stack contents, and checking for overflow and underflow conditions. The example usage demonstrates how to use the Stack class.

Task-2 – (Queue Design)

- **Task:** Use AI assistance to create a Queue data structure following FIFO principles

Expected Output:

- A complete Queue implementation including enqueue, dequeue, front element access, and size calculation

Prompt:

Design and implement a **Queue data structure in Python** that follows the **FIFO (First-In-First-Out)** principle. Use a class-based approach and include core operations such as **enqueue, dequeue, front element access, and size calculation**. Ensure the code is clean, well-structured, and includes basic documentation for each method.

Code:

```
class Queue:
```

```
    def __init__(self):
```

```
        self.queue = []
```

```
    def enqueue(self, item):
```

```
        """Adds an item to the end of the queue."""
```

```
        self.queue.append(item)
```

```
    def dequeue(self):
```

```
        """Removes and returns the item at the front of the
        queue. Raises an exception if the queue is empty."""
```

```
        if self.is_empty():
            raise Exception("Queue underflow: cannot remove
            item, queue is empty.")
        return self.queue.pop(0)
```

```
def front(self):
```

```
    """Returns the item at the front of the queue without
    removing it. Raises an exception if the queue is empty."""
```

```
    if self.is_empty():
        raise Exception("Queue underflow: cannot access
        front item, queue is empty.")
    return self.queue[0]
```

```
def size(self):
```

```
    """Returns the number of items in the queue."""
    return len(self.queue)
```

```
def is_empty(self):
```

```
    """Checks if the queue is empty."""
    return len(self.queue) == 0
```

```

11.2.2.py > ...
1  #Design and implement a Queue data structure in Python that follows the FIFO (First-In-First-Out) principle.
2  #Use a class-based approach and include core operations such as enqueue, dequeue, front element access, and size calculation.
3  #Ensure the code is clean, well-structured, and includes basic documentation for each method.
4
5  class Queue:
6      def __init__(self):
7          self.queue = []
8
9      def enqueue(self, item):
10         """Adds an item to the end of the queue."""
11         self.queue.append(item)
12
13     def dequeue(self):
14         """Removes and returns the item at the front of the queue. Raises an exception if the queue is empty."""
15         if self.is_empty():
16             raise Exception("Queue underflow: cannot remove item, queue is empty.")
17         return self.queue.pop(0)
18
19     def front(self):
20         """Returns the item at the front of the queue without removing it. Raises an exception if the queue is empty."""
21         if self.is_empty():
22             raise Exception("Queue underflow: cannot access front item, queue is empty.")
23         return self.queue[0]
24
25     def size(self):
26         """Returns the number of items in the queue."""
27         return len(self.queue)
28
29     def is_empty(self):
30         """Checks if the queue is empty."""
31         return len(self.queue) == 0
32

```

Output:

```

33  # Example usage:
34  if __name__ == "__main__":
35      queue = Queue()
36      queue.enqueue(1)
37      queue.enqueue(2)
38      queue.enqueue(3)
39      print("Front item:", queue.front())
40      print("Queue size:", queue.size())
41      print("Dequeued item:", queue.dequeue())
42      print("Front item after dequeue:", queue.front())
43      print("Is queue empty?", queue.is_empty())

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

assited coding/ai_assi_codes/11.2/11.2.2.py"
Front item: 1
Queue size: 3
Dequeued item: 1
Front item after dequeue: 2
Is queue empty? False
PS D:\Subjects\ai assited coding\ai_assi_codes\11.2>

```


Explanation:

The Queue data structure was designed using a class-based approach in Python, with the help of AI assistance to ensure that the implementation is clean, well-structured, and includes basic documentation for each method. The Queue follows the FIFO (First-In-First-Out) principle, which means that the first item added to the queue will be the first one to be removed.

Task Description -3 –(Singly Linked List Construction)

- **Task:** Utilize AI to build a singly linked list supporting insertion and traversal.

Expected Output:

- Correctly functioning linked list with node creation, insertion logic, and display functionality.

Prompt:

#Design and implement a **Singly Linked List in Python** using classes for the node and the list. Include functionality for **node creation, insertion of new nodes, and traversal to display all elements**. Ensure the implementation follows clear structure and basic documentation.

Code:

class Node:

def __init__(self, data):

"""Initializes a node with the given data and sets the next pointer to None."""

```

        self.data = data
        self.next = None
class SinglyLinkedList:
    def __init__(self):
        """Initializes an empty singly linked list with head set to
        None."""
        self.head = None

    def insert(self, data):
        """Inserts a new node with the given data at the end of
        the list."""
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            return
        last_node = self.head
        while last_node.next:
            last_node = last_node.next
        last_node.next = new_node

    def display(self):
        """Traverses the list and prints the data of each node."""
        current_node = self.head

```

```
while current_node:
    print(current_node.data, end=' ')
    current_node = current_node.next
print()
```

Example usage:

```
if __name__ == "__main__":
    linked_list = SinglyLinkedList()
    linked_list.insert(1)
    linked_list.insert(2)
    linked_list.insert(3)
    print("Singly Linked List contents:")
    linked_list.display()
```

```

11.2.3.py > ...
1  #Design and implement a Singly Linked List in Python using classes for the node and the list.
2  #Include functionality for node creation, insertion of new nodes, and traversal to display all elements.
3  #Ensure the implementation follows clear structure and basic documentation.
4
5  class Node:
6      def __init__(self, data):
7          """Initializes a node with the given data and sets the next pointer to None."""
8          self.data = data
9          self.next = None
10
11 class SinglyLinkedList:
12     def __init__(self):
13         """Initializes an empty singly linked list with head set to None."""
14         self.head = None
15
16     def insert(self, data):
17         """Inserts a new node with the given data at the end of the list."""
18         new_node = Node(data)
19         if self.head is None:
20             self.head = new_node
21             return
22         last_node = self.head
23         while last_node.next:
24             last_node = last_node.next
25         last_node.next = new_node
26
27     def display(self):
28         """Traverses the list and prints the data of each node."""
29         current_node = self.head
30         while current_node:
31             print(current_node.data, end=' ')
32             current_node = current_node.next
33         print()

```

Output:

```

32  # Example usage:
33  if __name__ == "__main__":
34      linked_list = SinglyLinkedList()
35      linked_list.insert(1)
36      linked_list.insert(2)
37      linked_list.insert(3)
38      print("Singly Linked List contents:")
39      linked_list.display()

```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

Is queue empty? False

PS D:\Subjects\ai assisted coding\ai_assi_codes\11.2> & C:/Users/ai_assi_codes/11.2/11.2.3.py

Singly Linked List contents:

1 2 3

PS D:\Subjects\ai assisted coding\ai_assi_codes\11.2>

Explanation:

The singly linked list was constructed using AI assistance by defining two classes: `Node` and `SinglyLinkedList`. The `Node` class represents each individual element in the list, containing a data field to store the value and a next pointer to reference the next node in the list. The `SinglyLinkedList` class manages the overall structure of the list, starting with an empty head. The insertion process involves creating a new node and adding it to the end of the list by traversing from the head to the last node. The traversal method allows us to display all elements in the list by iterating through each node until we reach the end (where the next pointer is None). This structure allows for efficient management of the list while maintaining a clear and organized codebase.

Task Description -4 – (Binary Search Tree Operations)

- **Task:** Implement a Binary Search Tree with AI support focusing on insertion and traversal.

Expected Output:

- BST program with correct node insertion and in-order traversal output.

Prompt:

#Design and implement a **Binary Search Tree (BST)** in **Python** using classes for nodes and the tree structure. Include functionality for **inserting nodes according to BST rules** and

performing **in-order traversal** to display the elements in sorted order. Ensure the code is clear and properly structured.

Code:

#Design and implement a Binary Search Tree (BST) in Python using classes for nodes and the tree structure.

#Include functionality for inserting nodes according to BST rules and performing in-order traversal

#to display the elements in sorted order. Ensure the code is clear and properly structured.

```
class TreeNode:
```

```
    def __init__(self, key):
```

```
        """Initializes a tree node with the given key and sets left and right pointers to None."""
```

```
        self.left = None
```

```
        self.right = None
```

```
        self.val = key
```

```
class BinarySearchTree:
```

```
    def __init__(self):
```

```
        """Initializes an empty binary search tree with root set to None."""
```

```
        self.root = None
```

```

def insert(self, key):
    """Inserts a new node with the given key into the BST
    according to BST rules."""
    if self.root is None:
        self.root = TreeNode(key)
    else:
        self._insert_recursive(self.root, key)

def _insert_recursive(self, node, key):
    """Helper method to recursively insert a new node into
    the BST."""
    if key < node.val:
        if node.left is None:
            node.left = TreeNode(key)
        else:
            self._insert_recursive(node.left, key)
    else:
        if node.right is None:
            node.right = TreeNode(key)
        else:
            self._insert_recursive(node.right, key)

def in_order_traversal(self):

```

```
        """Performs in-order traversal of the BST and returns a
list of elements in sorted order."""
```

```
    return self._in_order_recursive(self.root)
```

```
def _in_order_recursive(self, node):
```

```
    """Helper method to recursively perform in-order
traversal."""
```

```
    elements = []
```

```
    if node:
```

```
        elements += self._in_order_recursive(node.left)
```

```
        elements.append(node.val)
```

```
        elements += self._in_order_recursive(node.right)
```

```
    return elements
```

```
# Example usage:
```

```
if __name__ == "__main__":
```

```
    bst = BinarySearchTree()
```

```
    bst.insert(5)
```

```
    bst.insert(3)
```

```
    bst.insert(7)
```

```
    bst.insert(2)
```

```
    bst.insert(4)
```

```
    bst.insert(6)
```



```
bst.insert(8)
```

```
print("In-order traversal of the BST:",  
bst.in_order_traversal())
```

Output:

```
49 # Example usage:  
50 if __name__ == "__main__":  
51     bst = BinarySearchTree()  
52     bst.insert(5)  
53     bst.insert(3)  
54     bst.insert(7)  
55     bst.insert(2)  
56     bst.insert(4)  
57     bst.insert(6)  
58     bst.insert(8)  
59  
60     print("In-order traversal of the BST:", bst.in_order_traversal())
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
ai_assi_codes/11.2/11.2.4.py"  
In-order traversal of the BST: [2, 3, 4, 5, 6, 7, 8]  
PS D:\Subjects\ai assisted coding\ai_assi_codes\11.2> █
```

Explanation:

The Binary Search Tree (BST) was implemented using two classes: `TreeNode` and `BinarySearchTree`. The `TreeNode` class represents each node in the tree, containing a key value and pointers to the left and right child nodes. The `BinarySearchTree` class manages the overall structure of the tree, including the root node and methods for inserting nodes and performing in-order traversal.

Task Description -5 – (Hash Table Implementation)

- **Task:** Create a hash table using AI with collision handling

Expected Output:

- Hash table supporting insert, search, and delete using chaining or open

Prompt:

#Design and implement a **Hash Table in Python** using AI assistance. The implementation should support **insert, search, and delete operations**, and include a **collision handling technique** such as chaining or open addressing. Ensure the code is structured clearly with basic documentation for each method.

Code:

#Design and implement a Hash Table in Python using AI assistance. The implementation should support insert, search, and delete

#operations, and include a collision handling technique such as chaining or open addressing. Ensure the code is structured #clearly with basic documentation for each method.

```
class HashTable:
```

```
    def __init__(self, size=10):
```

```
        """Initializes a hash table with a specified size and an empty list for chaining."""
```

```
self.size = size
```

```
self.table = [[] for _ in range(size)]
```

```
def _hash(self, key):
```

```
    """Generates a hash for the given key using the built-in  
hash function and modulo operator."""
```

```
    return hash(key) % self.size
```

```
def insert(self, key, value):
```

```
    """Inserts a key-value pair into the hash table. Handles  
collisions using chaining."""
```

```
    index = self._hash(key)
```

```
    # Check if the key already exists and update it
```

```
    for i, (k, v) in enumerate(self.table[index]):
```

```
        if k == key:
```

```
            self.table[index][i] = (key, value) # Update existing  
key
```

```
            return
```

```
    # If key does not exist, add new key-value pair
```

```
    self.table[index].append((key, value))
```

```
def search(self, key):
```

```
    """Searches for a value by its key in the hash table.  
Returns the value if found, else returns None."""
```

```
index = self._hash(key)
for k, v in self.table[index]:
    if k == key:
        return v
return None
```

```
def delete(self, key):
    """Deletes a key-value pair from the hash table. Returns
    True if deletion was successful, else False."""
    index = self._hash(key)
    for i, (k, v) in enumerate(self.table[index]):
        if k == key:
            del self.table[index][i] # Remove the key-value pair
            return True
    return False
```

Example usage:

```
if __name__ == "__main__":
    hash_table = HashTable()
    hash_table.insert("name", "Alice")
    hash_table.insert("age", 30)
    hash_table.insert("city", "New York")
```

```
print("Search for 'name':", hash_table.search("name"))
```

```
print("Search for 'age':", hash_table.search("age"))
```

```
print("Search for 'city':", hash_table.search("city"))
```

```
print("Delete 'age':", hash_table.delete("age"))
```

```
print("Search for 'age' after deletion:",  
hash_table.search("age"))
```

```
1  #Design and implement a Hash Table in Python using AI assistance. The implementation should support insert, search, and delete
2  #operations, and include a collision handling technique such as chaining or open addressing. Ensure the code is structured
3  #clearly with basic documentation for each method.
4
5  class HashTable:
6      def __init__(self, size=10):
7          """Initializes a hash table with a specified size and an empty list for chaining."""
8          self.size = size
9          self.table = [[] for _ in range(size)]
10
11     def _hash(self, key):
12         """Generates a hash for the given key using the built-in hash function and modulo operator."""
13         return hash(key) % self.size
14
15     def insert(self, key, value):
16         """Inserts a key-value pair into the hash table. Handles collisions using chaining."""
17         index = self._hash(key)
18         # Check if the key already exists and update it
19         for i, (k, v) in enumerate(self.table[index]):
20             if k == key:
21                 self.table[index][i] = (key, value) # Update existing key
22                 return
23         # If key does not exist, add new key-value pair
24         self.table[index].append((key, value))
25
26     def search(self, key):
27         """Searches for a value by its key in the hash table. Returns the value if found, else returns None."""
28         index = self._hash(key)
29         for k, v in self.table[index]:
30             if k == key:
31                 return v
32         return None
33
34     def delete(self, key):
35         """Deletes a key-value pair from the hash table. Returns True if deletion was successful, else False."""
36         index = self._hash(key)
37         for i, (k, v) in enumerate(self.table[index]):
38             if k == key:
39                 del self.table[index][i] # Remove the key-value pair
40                 return True
41         return False
```

Output:

```
11.2.5.py > HashTable > delete
42
43 # Example usage:
44 if __name__ == "__main__":
45     hash_table = HashTable()
46     hash_table.insert("name", "Alice")
47     hash_table.insert("age", 30)
48     hash_table.insert("city", "New York")
49
50     print("Search for 'name':", hash_table.search("name"))
51     print("Search for 'age':", hash_table.search("age"))
52     print("Search for 'city':", hash_table.search("city"))
53
54     print("Delete 'age':", hash_table.delete("age"))
55     print("Search for 'age' after deletion:", hash_table.search("age"))

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

ai_assi_codes/11.2/11.2.5.py"
Search for 'name': Alice
Search for 'age': 30
Search for 'city': New York
Delete 'age': True
Delete 'age': True
Search for 'age' after deletion: None
PS D:\Subjects\ai assisted coding\ai_assi_codes\11.2>

Delete 'age': True
Search for 'age' after deletion: None
PS D:\Subjects\ai assisted coding\ai_assi_codes\11.2>

Delete 'age': True
Search for 'age' after deletion: None
Search for 'age' after deletion: None
PS D:\Subjects\ai assisted coding\ai_assi_codes\11.2> 
```

Explanation:

The hash table was implemented using a list of lists (chaining) to handle collisions. The hash function generates an index for a given key by using Python's built-in `hash` function and the modulo operator with the size of the table. This index determines where the key-value pair will be stored in the table. If multiple keys hash to the same index, they are stored in a list at that index (chaining). The `insert` method adds a key-value pair to the appropriate index, updating the value if the key already exists. The `search` method looks for a key in the list at the hashed index and returns its value if found. The `delete` method removes a key-value pair from the list at the hashed index if it exists. This approach allows for efficient handling of collisions while maintaining the core functionality of a hash table.